

```
import os
import sys
import datetime

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import tkinter as tk
from tkinter import ttk, messagebox
import customtkinter as ctk
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib
import matplotlib.pyplot as plt
matplotlib.use("TkAgg")

from desktop_app.services.data_manager import DataManager

# — Theme


---



ctk.set_appearance_mode("Dark")
ctk.set_default_color_theme("green")

# — Color Palette


---



BG = "#0a0f1e"      # Deep navy background
SIDEBAR_BG = "#0d1627"  # Sidebar darker navy
CARD_BG = "#111c35"    # Card dark blue
```

```
CARD_BG2      = "#0f1b30"      # Slightly different card
ACCENT        = "#00d4aa"      # Teal accent
ACCENT2       = "#7c3aed"      # Purple accent
ACCENT3       = "#f59e0b"      # Amber accent
ACCENT4       = "#ef4444"      # Red accent
TEXT_PRI      = "#f0f6ff"      # Primary white text
TEXT_SEC      = "#8aa0c0"      # Muted secondary text
BORDER        = "#1e3a5f"      # Subtle border

CATS          = ["plastic", "paper", "glass", "metal", "organic"]
CAT_CLR       = {"plastic": "#3b82f6", "paper": "#f59e0b",
                  "glass":   "#10b981", "metal":   "#64748b", "organic": "#a855f7"}
```

```
# ——— TTK Table Styling
```

```
def apply_table_style():
    style = ttk.Style()
    style.theme_use("clam")
    style.configure("Custom.Treeview",
                    background=CARD_BG,
                    foreground=TEXT_PRI,
                    rowheight=32,
                    fieldbackground=CARD_BG,
                    bordercolor=BORDER,
                    borderwidth=0,
                    font=("Segoe UI", 11))
```

```

)

style.configure("Custom.Treeview.Heading",
    background="#1e3a5f",
    foreground=ACCENT,
    font=("Segoe UI", 11, "bold"),
    relief="flat",
    borderwidth=0
)

style.map("Custom.Treeview",
    background=[("selected", "#1a3a5c")],
    foreground=[("selected", ACCENT)]
)

style.configure("Custom.Vertical.TScrollbar",
    background=CARD_BG, troughcolor=SIDEBAR_BG,
    arrowcolor=ACCENT, bordercolor=BORDER
)

#
=====

class SmartWasteApp(ctk.CTk):
    def __init__(self):
        super().__init__()

        self.title("🗑️ SmartWaste – Smart Waste & Recycling Management System")
        self.geometry("1280x800")
        self.minsize(1100, 720)
        self.configure(fg_color=BG)

```

```
    apply_table_style()

    self.data_manager = DataManager()

    self._active_btn = None

    self._build_layout()

    self.show_dashboard()

# ——— Layout Shell


---



def _build_layout(self):

    self.sidebar = tk.Frame(self, bg=SIDEBAR_BG, width=230)

    self.sidebar.pack(side="left", fill="y")

    self.sidebar.pack_propagate(False)

    self.main = tk.Frame(self, bg=BG)

    self.main.pack(side="left", fill="both", expand=True)

    self._build_sidebar()

def _build_sidebar(self):

    # Logo

    logo_frame = tk.Frame(self.sidebar, bg=SIDEBAR_BG)

    logo_frame.pack(fill="x", pady=(25, 5), padx=18)

    tk.Label(logo_frame, text="🔄", font=("Segoe UI", 28), bg=SIDEBAR_BG,
fg=ACCENT).pack(side="left")

    txt_f = tk.Frame(logo_frame, bg=SIDEBAR_BG)
```

```

txt_f.pack(side="left", padx=8)

tk.Label(txt_f, text="SmartWaste", font=("Segoe UI", 15, "bold"),
         bg=SIDEBAR_BG, fg=TEXT_PRI).pack(anchor="w")

tk.Label(txt_f, text="Management System", font=("Segoe UI", 9),
         bg=SIDEBAR_BG, fg=TEXT_SEC).pack(anchor="w")

tk.Frame(self.sidebar, bg=BORDER, height=1).pack(fill="x", padx=18,
pady=16)

# Nav buttons

self._nav_btns = {}

nav_items = [
    ("dashboard", " 🏠 Dashboard", self.show_dashboard),
    ("records", " 📁 Records", self.show_records),
    ("add", " + Add Collection", self.show_add_form),
    ("suggestions", " 🧠 Smart Suggestions", self.show_suggestions),
    ("inspector", " 🔍 Data Structures", self.show_ds_inspector),
]

for key, label, cmd in nav_items:
    btn = self._nav_btn(self.sidebar, label, cmd, key)
    btn.pack(fill="x", padx=12, pady=3)
    self._nav_btns[key] = btn

tk.Frame(self.sidebar, bg=BORDER, height=1).pack(fill="x", padx=18,
pady=16)

# Undo Button

```

```

undo_btn = tk.Button(
    self.sidebar, text="↔ Undo Last Action",
    bg="#2a1215", fg="#ef4444", activebackground="#3d1a1f",
    activeforeground="#ef4444",
    font=("Segoe UI", 12), bd=0, padx=8, pady=10,
    anchor="w", cursor="hand2", relief="flat",
    command=self.undo_action
)
undo_btn.pack(fill="x", padx=12, pady=2)

# Footer DS Labels
footer = tk.Frame(self.sidebar, bg=SIDEBAR_BG)
footer.pack(side="bottom", fill="x", padx=18, pady=20)
tk.Label(footer, text="Active Data Structures", font=("Segoe UI", 9,
"bold"),
        bg=SIDEBAR_BG, fg=TEXT_SEC).pack(anchor="w")

for ds in ["• Doubly Linked List", "• Stack (LIFO Undo)", "• Priority
Queue (Heap)",
          "• Hash Table O(1)", "• Binary Search Tree"]:
```

```

    tk.Label(footer, text=ds, font=("Segoe UI", 9),
            bg=SIDEBAR_BG, fg="#475569").pack(anchor="w")

def _nav_btn(self, parent, text, cmd, key):
    btn = tk.Button(
        parent, text=text, anchor="w",
        bg=SIDEBAR_BG, fg=TEXT_SEC, activebackground="#152040",
        activeforeground=ACCENT, font=("Segoe UI", 12),
        bd=0, padx=10, pady=10, relief="flat", cursor="hand2",

```

```

        command=lambda: self._nav_click(key, cmd)

    )

    return btn

def _nav_click(self, key, cmd):

    # Deselect old

    if self._active_btn:

        self._nav_btns[self._active_btn].configure(bg=SIDEBAR_BG,
fg=TEXT_SEC)

    # Select new

    self._active_btn = key

    self._nav_btns[key].configure(bg="#152040", fg=ACCENT)

    cmd()

def clear_main(self):

    for w in self.main.winfo_children():

        w.destroy()

def _page_header(self, title, subtitle=""):

    hdr = tk.Frame(self.main, bg=BG)

    hdr.pack(fill="x", padx=30, pady=(25, 10))

    tk.Label(hdr, text=title, font=("Segoe UI", 22, "bold"),

        bg=BG, fg=TEXT_PRI).pack(anchor="w")

    if subtitle:

        tk.Label(hdr, text=subtitle, font=("Segoe UI", 11),

            bg=BG, fg=TEXT_SEC).pack(anchor="w")

```

```

        tk.Frame(self.main, bg=BORDER, height=1).pack(fill="x", padx=30, pady=(0,
5))

def _card(self, parent, **kwargs):
    return tk.Frame(parent, bg=CARD_BG, **kwargs)

#
=====

# DASHBOARD
#
=====

def show_dashboard(self):
    self.clear_main()

    records = self.data_manager.get_all_records()

    total_kg = sum(r["total_kg"] for r in records)

    avg_rec = (sum(r["recyclable_pct"] for r in records) / len(records)) if
records else 0

    unique_areas = len({r["area"] for r in records})

    self._page_header("Dashboard", f"Overview for
{datetime.date.today().strftime('%B %d, %Y')}")

# ——— Metric Cards
=====

row = tk.Frame(self.main, bg=BG)

row.pack(fill="x", padx=30, pady=8)

metrics = [

```



```

        ("📦", "Total Collected", f"{total_kg:.1f} kg", ACCENT),
        ("♻️", "Avg Recycling Rate", f"{avg_rec:.1f}%", "#3b82f6"),
        ("🚚", "Total Pickups", str(len(records)), ACCENT3),
        ("🌍", "Active Zones", str(unique_areas), ACCENT2),
    ]

    for icon, label, val, color in metrics:
        card = tk.Frame(row, bg=CARD_BG, bd=0, relief="flat")
        card.pack(side="left", fill="both", expand=True, padx=6, ipady=14,
ipadx=10)

        tk.Label(card, text=icon, font=("Segoe UI Emoji", 20),
                bg=CARD_BG, fg=color).pack(anchor="w", padx=15, pady=(14,
4))

        tk.Label(card, text=val, font=("Segoe UI", 22, "bold"),
                bg=CARD_BG, fg=color).pack(anchor="w", padx=15)

        tk.Label(card, text=label, font=("Segoe UI", 10),
                bg=CARD_BG, fg=TEXT_SEC).pack(anchor="w", padx=15, pady=(2,
14))

# — Charts

charts_card = tk.Frame(self.main, bg=CARD_BG)
charts_card.pack(fill="both", expand=True, padx=30, pady=(8, 25))

cat_totals = {c: 0.0 for c in CATS}
area_totals = {}

for r in records:
    for cat in CATS:
        cat_totals[cat] += r.get("waste", {}).get(cat, 0)

```

```

        a = r.get("area", "Unknown")

        area_totals[a] = area_totals.get(a, 0) + r["total_kg"]

    plt.rcParams.update({"figure.facecolor": CARD_BG, "axes.facecolor":
CARD_BG,

                        "text.color": TEXT_PRI, "axes.labelcolor": TEXT_SEC,

                        "xtick.color": TEXT_SEC, "ytick.color": TEXT_SEC})

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 3.8),
facecolor=CARD_BG,

                                gridspec_kw={"wspace": 0.35})

    clr = [CAT_CLR[c] for c in CATS]
    vals = [cat_totals[c] for c in CATS]
    labels = [c.capitalize() for c in CATS]

    wedges, texts, autotexts = ax1.pie(

        vals, labels=labels, colors=clr, autopct='%1.0f%%',

        pctdistance=0.78, startangle=140,

        wedgeprops=dict(linewidth=2, edgecolor=CARD_BG)

    )

    for t in texts: t.set_fontsize(9); t.set_color(TEXT_SEC)

    for a in autotexts: a.set_fontsize(9); a.set_color(TEXT_PRI);
a.set_fontweight("bold")

    ax1.set_title("Category Breakdown", fontsize=12, fontweight="bold",
color=TEXT_PRI, pad=12)

    bars = ax2.bar(list(area_totals.keys()), list(area_totals.values()),

```

```

        color=ACCENT, edgecolor="none", width=0.55)

    for bar in bars:

        h = bar.get_height()

        ax2.text(bar.get_x() + bar.get_width()/2, h + 0.3,
                  f"{h:.0f}kg", ha="center", va="bottom", fontsize=8,
color=TEXT_SEC)

        ax2.set_title("Waste per Zone (kg)", fontsize=12, fontweight="bold",
color=TEXT_PRI, pad=12)

        ax2.tick_params(axis='x', labelsize=8, rotation=20)

        for spine in ax2.spines.values():

            spine.set_color(BORDER)

        ax2.set_facecolor(CARD_BG2)

        ax2.grid(axis="y", color=BORDER, linestyle="--", linewidth=0.5,
alpha=0.5)


    canvas = FigureCanvasTkAgg(fig, master=charts_card)

    canvas.draw()

    canvas.get_tk_widget().pack(fill="both", expand=True, padx=5, pady=5)

    plt.close(fig)


#


---




---


# RECORDS
#


---




---



def show_records(self):

    self.clear_main()

```

```

        self._page_header("Records Manager", "Browse, search and review all waste
collection records")

# Search Bar

sb = tk.Frame(self.main, bg=BG)

sb.pack(fill="x", padx=30, pady=(6, 8))

self._search_var = tk.StringVar()

search_bg = tk.Frame(sb, bg=CARD_BG, padx=8, pady=4)

search_bg.pack(side="left")

tk.Label(search_bg, text="🔍", bg=CARD_BG, fg=ACCENT, font=("Segoe UI",
12)).pack(side="left")

entry = tk.Entry(search_bg, textvariable=self._search_var, bg=CARD_BG,
                  fg=TEXT_PRI, insertbackground=ACCENT, relief="flat",
                  font=("Segoe UI", 12), width=35)

entry.pack(side="left", padx=6, pady=2)

entry.bind("<Return>", lambda e: self._do_search())

self._search_btn("Search", self._do_search, ACCENT,
"#0d1627").pack(side="left", padx=8)

self._search_btn("Clear", self._clear_search, CARD_BG,
TEXT_SEC).pack(side="left")

# Table

tbl_card = tk.Frame(self.main, bg=CARD_BG, bd=0)

tbl_card.pack(fill="both", expand=True, padx=30, pady=(0, 25))

```

```

        cols =
("id","resident","area","date","plastic","paper","glass","metal","organic","total",
,"recycle%")

        self.tree = ttk.Treeview(tbl_card, columns=cols, show="headings",
                                style="Custom.Treeview", selectmode="browse")

        widths = [70,170,160,100,80,80,75,75,90,85,95]

        heads =
["ID","Resident","Zone","Date","Plastic","Paper","Glass","Metal","Organic","Total",
,"♻ Rate"]

        for col, head, w in zip(cols, heads, widths):

            self.tree.heading(col, text=head)

            self.tree.column(col, width=w, anchor="center" if w < 100 else "w",
stretch=False)

        sb2 = ttk.Scrollbar(tbl_card, orient="vertical", command=self.tree.yview,
                            style="Custom.Vertical.TScrollbar")

        self.tree.configure(yscroll=sb2.set)

        self.tree.pack(side="left", fill="both", expand=True, padx=2, pady=2)

        sb2.pack(side="right", fill="y", pady=2)

        self._fill_table(self.data_manager.get_all_records())

def _search_btn(self, text, cmd, bg, fg):

    return tk.Button(self.main, text=text, bg=bg, fg=fg,
                    activebackground=ACCENT, activeforeground=BG,
                    font=("Segoe UI", 11), bd=0, padx=14, pady=6,
                    relief="flat", cursor="hand2", command=cmd)

```

```

def _fill_table(self, records):

    for item in self.tree.get_children():

        self.tree.delete(item)

    for r in records:

        w = r.get("waste", {})

        self.tree.insert("", "end", values=(

            r["collection_id"], r["resident"], r["area"], r["date"],

            f"{w.get('plastic',0):.1f}", f"{w.get('paper',0):.1f}",

            f"{w.get('glass',0):.1f}", f"{w.get('metal',0):.1f}",

            f"{w.get('organic',0):.1f}", f"{r['total_kg']:.1f}",

            f"{r['recyclable_pct']:.1f}%"

        ))

```

```

def _do_search(self):

    q = self._search_var.get().strip()

    self._fill_table(self.data_manager.search_records(q))

```

```

def _clear_search(self):

    self._search_var.set("")

    self._fill_table(self.data_manager.get_all_records())

```

```

#

```

```

# ADD COLLECTION FORM

```

```

#

```

```

def show_add_form(self):

    self.clear_main()

    self._page_header("Add New Collection", "Log a waste pickup – data is
stored across all DS structures")

    # Scroll wrapper

    outer = tk.Frame(self.main, bg=BG)

    outer.pack(fill="both", expand=True, padx=30, pady=(0, 25))

    canvas = tk.Canvas(outer, bg=BG, bd=0, highlightthickness=0)

    canvas.pack(side="left", fill="both", expand=True)

    sc = ttk.Scrollbar(outer, orient="vertical", command=canvas.yview)

    sc.pack(side="right", fill="y")

    canvas.configure(yscrollcommand=sc.set)

    inner = tk.Frame(canvas, bg=BG)

    win_id = canvas.create_window((0, 0), window=inner, anchor="nw")

    def _on_resize(e):

        canvas.itemconfig(win_id, width=e.width)

        canvas.bind("<Configure>", _on_resize)

        inner.bind("<Configure>", lambda e:
canvas.configure(scrollregion=canvas.bbox("all")))

    # — Two-column form

    top_row = tk.Frame(inner, bg=BG)

```

```

top_row.pack(fill="x", pady=8)

left_card  = tk.Frame(top_row, bg=CARD_BG, padx=22, pady=18)
right_card = tk.Frame(top_row, bg=CARD_BG, padx=22, pady=18)
left_card.pack(side="left", fill="both", expand=True, padx=(0, 8))
right_card.pack(side="left", fill="both", expand=True, padx=(8, 0))

# - Left: Identity Fields -

self._section_hdr(left_card, "Collection Identity")
records = self.data_manager.get_all_records()
next_id = f"WC{len(records) + 1:03d}"

self.e_id  = self._form_field(left_card, "Collection ID",      next_id)
self.e_res = self._form_field(left_card, "Resident / Collector Name", "")
self.e_area = self._form_optionmenu(left_card, "Municipal Zone",
    ["Zone A (Block 1)", "Zone B (Block 2)", "Zone C (Block 3)",
    "Zone D (Market)", "Zone E (Industrial)"])

self.e_date = self._form_field(left_card, "Collection Date (YYYY-MM-DD)",
    str(datetime.date.today()))

self.e_notes = self._form_field(left_card, "Notes / Remarks (optional)",
    "")

# - Right: Waste Breakdown -

self._section_hdr(right_card, "Waste Breakdown (kg)")

tk.Label(right_card, text="Enter the weight collected per category
below:",
    font=("Segoe UI", 10), bg=CARD_BG, fg=TEXT_SEC).pack(anchor="w",
    pady=(0, 10))

```



```

self.waste_entries = {}

for cat in CATS:

    row = tk.Frame(right_card, bg=CARD_BG)

    row.pack(fill="x", pady=5)

    clr_dot = tk.Frame(row, bg=CAT_CLR[cat], width=10, height=10)

    clr_dot.pack(side="left", padx=(0, 8), pady=4)

    tk.Label(row, text=cat.capitalize(), font=("Segoe UI", 12),
              bg=CARD_BG, fg=TEXT_PRI, width=9,
anchor="w").pack(side="left")

    ent = tk.Entry(row, bg=CARD_BG2, fg=TEXT_PRI,
insertbackground=ACCENT,

                    relief="flat", font=("Segoe UI", 12), width=10,

                    highlightthickness=1, highlightbackground=BORDER,

                    highlightcolor=ACCENT)

    ent.insert(0, "0.0")

    ent.pack(side="left", padx=8, ipady=5)

    tk.Label(row, text="kg", font=("Segoe UI", 10),
              bg=CARD_BG, fg=TEXT_SEC).pack(side="left")

    self.waste_entries[cat] = ent


# – Live Total Preview –

self.total_var = tk.StringVar(value="Total: 0.0 kg")

total_lbl = tk.Label(right_card, textvariable=self.total_var,

                      font=("Segoe UI", 14, "bold"), bg=CARD_BG,
fg=ACCENT)

```

```

total_lbl.pack(anchor="e", pady=(14, 0))

def _update_total(*_):
    total = 0.0
    for cat, ent in self.waste_entries.items():
        try:
            total += float(ent.get() or 0)
        except ValueError:
            pass
    self.total_var.set(f"Total: {total:.2f} kg")

for ent in self.waste_entries.values():
    ent.bind("<KeyRelease>", _update_total)

# - Submit Button -
submit_frame = tk.Frame(inner, bg=BG)
submit_frame.pack(fill="x", pady=12)
tk.Button(
    submit_frame,
    text="  Save Collection Record",
    bg=ACCENT, fg=BG,
    activebackground="#00b390", activeforeground=BG,
    font=("Segoe UI", 13, "bold"),
    bd=0, padx=28, pady=12, relief="flat", cursor="hand2",
    command=self._submit_record
).pack(side="left")
tk.Button(

```

```

        submit_frame, text=" 🗑️ Clear Form",
        bg=CARD_BG, fg=TEXT_SEC, activebackground=BORDER,
        activeforeground=TEXT_PRI, font=("Segoe UI", 12),
        bd=0, padx=20, pady=12, relief="flat", cursor="hand2",
        command=self.show_add_form
    ).pack(side="left", padx=12)

def _section_hdr(self, parent, text):
    tk.Label(parent, text=text, font=("Segoe UI", 13, "bold"),
             bg=CARD_BG, fg=ACCENT).pack(anchor="w", pady=(0, 12))
    tk.Frame(parent, bg=BORDER, height=1).pack(fill="x", pady=(0, 12))

def _form_field(self, parent, label, default):
    tk.Label(parent, text=label, font=("Segoe UI", 10),
             bg=CARD_BG, fg=TEXT_SEC).pack(anchor="w", pady=(8, 2))
    ent = tk.Entry(parent, bg=CARD_BG2, fg=TEXT_PRI, insertbackground=ACCENT,
                   relief="flat", font=("Segoe UI", 12),
                   highlightthickness=1, highlightbackground=BORDER,
highlightcolor=ACCENT)
    ent.insert(0, default)
    ent.pack(fill="x", ipady=7, pady=(0, 4))
    return ent

def _form_optionmenu(self, parent, label, options):
    tk.Label(parent, text=label, font=("Segoe UI", 10),
             bg=CARD_BG, fg=TEXT_SEC).pack(anchor="w", pady=(8, 2))
    var = tk.StringVar(value=options[0])

```

```

om = tk.OptionMenu(parent, var, *options)

om.configure(bg=CARD_BG2, fg=TEXT_PRI, activebackground=BORDER,
             activeforeground=ACCENT, relief="flat",
             font=("Segoe UI", 12), bd=0, highlightthickness=0)

om["menu"].configure(bg=CARD_BG, fg=TEXT_PRI, activebackground="#1e3a5f",
                    activeforeground=ACCENT, font=("Segoe UI", 11))

om.pack(fill="x", pady=(0, 4), ipady=4)

return var


def _submit_record(self):

    cid      = self.e_id.get().strip()

    resident = self.e_res.get().strip()

    area     = self.e_area.get()

    date     = self.e_date.get().strip()

    if not cid:

        messagebox.showerror("Validation", "Collection ID is required!")

        return

    if not resident:

        messagebox.showerror("Validation", "Resident / Collector name is
required!")

        return

    waste_dict = {}

    for cat, ent in self.waste_entries.items():

        try:

            waste_dict[cat] = max(0.0, float(ent.get()))

```

```

        except ValueError:

            waste_dict[cat] = 0.0

    if sum(waste_dict.values()) == 0:

        messagebox.showwarning("No Waste Data", "Please enter at least one
waste category amount.")

        return

    record = {"collection_id": cid, "resident": resident,

              "area": area, "date": date, "waste": waste_dict}

    self.data_manager.add_record(record, record_undo=True, save_to_file=True)

    messagebox.showinfo("Success ✔",

                        f"Record '{cid}' for {resident} saved!\n"

                        f"Total: {sum(waste_dict.values()):.2f} kg
collected.")

    self.show_records()

#
=====
=====

# SMART SUGGESTIONS

#
=====
=====

def show_suggestions(self):

    self.clear_main()

    self._page_header("Smart Suggestions", "AI-driven eco tips and municipal
recommendations based on your data")

```

```

records = self.data_manager.get_all_records()

cat_totals = {c: 0.0 for c in CATS}

for r in records:
    for cat in CATS:
        cat_totals[cat] += r.get("waste", {}).get(cat, 0)

total_waste = sum(cat_totals.values()) or 1

plastic_pct = cat_totals["plastic"] / total_waste * 100
organic_pct = cat_totals["organic"] / total_waste * 100
paper_pct = cat_totals["paper"] / total_waste * 100
metal_pct = cat_totals["metal"] / total_waste * 100
recyclable = (cat_totals["plastic"] + cat_totals["paper"] +
              cat_totals["glass"] + cat_totals["metal"]) /
total_waste * 100

avg_pickup = (sum(r["total_kg"] for r in records) / len(records)) if
records else 0

# Build suggestion cards
suggestions = []

if plastic_pct > 30:
    suggestions.append((" ", "High Plastic Volume Detected",
                       f"{plastic_pct:.1f}% of total waste is plastic. Launch a plastic-
free drive and "
                       "partner with local vendors to reduce single-use packaging.",
                       "#ef4444", "Critical"))

if organic_pct > 40:

```

```

        suggestions.append(("🌱", "Set Up Composting Units",
                               f"{organic_pct:.1f}% of waste is organic. Municipal compost bins
or biogas plants "
                               "could convert this to fertilizer or clean energy, reducing
landfill load by ~35%.",
                               "#a855f7", "High"))

    if recyclable < 50:
        suggestions.append(("♻️", "Boost Recycling Awareness",
                               f"Only {recyclable:.1f}% of waste is recyclable material. Run
educational campaigns "
                               "on proper waste segregation (colour-coded bins: Blue=Dry,
Green=Wet).",
                               "#f59e0b", "Medium"))

    if avg_pickup > 20:
        suggestions.append(("🚚", "Optimize Pickup Frequency",
                               f"Average pickup load is {avg_pickup:.1f} kg – exceeding
recommended 15 kg/household. "
                               "Increase collection rounds or deploy larger trucks in high-
density zones.",
                               "#3b82f6", "High"))

    if paper_pct > 20:
        suggestions.append(("📄", "Paper Recycling Program",
                               f"{paper_pct:.1f}% paper waste detected. Partner with recycling
mills – "
                               "every tonne of recycled paper saves 17 trees and 26,000 litres
of water.",
                               ACCENT, "Medium"))

```

```

if metal_pct > 15:
    suggestions.append(("🔍", "Metal Scrap Collection Drive",
        f"{metal_pct:.1f}% is metal waste. Organise quarterly scrap
drives. "
        "Recycling metal saves up to 95% energy compared to producing
from ore.",
        ACCENT3, "Medium"))

# General suggestions always shown
suggestions.append(("🏢", "Schedule Regular Audits",
    "Monthly waste data audits help identify area-wise trends early. "
    "Use the Dashboard Charts to compare zone performance and take
proactive action.",
    "#64748b", "General"))

suggestions.append(("➡️", "Digital Reporting & Resident Awareness",
    "Send monthly recycling reports to residents. Communities with
regular feedback "
    "reduce waste generation by an average of 22% within 6 months.",
    "#64748b", "General"))

# Render
scroll_outer = tk.Frame(self.main, bg=BG)
scroll_outer.pack(fill="both", expand=True, padx=30, pady=(0, 25))

canvas = tk.Canvas(scroll_outer, bg=BG, bd=0, highlightthickness=0)
canvas.pack(side="left", fill="both", expand=True)

```



```

sc = ttk.Scrollbar(scroll_outer, orient="vertical", command=canvas.yview)
sc.pack(side="right", fill="y")

canvas.configure(yscrollcommand=sc.set)

sframe = tk.Frame(canvas, bg=BG)

win = canvas.create_window((0, 0), window=sframe, anchor="nw")

canvas.bind("<Configure>", lambda e: canvas.itemconfig(win,
width=e.width))

sframe.bind("<Configure>", lambda e:
canvas.configure(scrollregion=canvas.bbox("all")))

PRIORITY_CLR = {"Critical": "#ef4444", "High": "#f59e0b",
                 "Medium": "#3b82f6", "General": "#64748b"}

for icon, title, desc, accent_col, priority in suggestions:

    card = tk.Frame(sframe, bg=CARD_BG, padx=0, pady=0)
    card.pack(fill="x", pady=7)

    # Color stripe on left

    stripe = tk.Frame(card, bg=accent_col, width=5)
    stripe.pack(side="left", fill="y")

    body = tk.Frame(card, bg=CARD_BG, padx=18, pady=16)
    body.pack(side="left", fill="both", expand=True)

    title_row = tk.Frame(body, bg=CARD_BG)
    title_row.pack(fill="x")

```

```
tk.Label(title_row, text=f"{icon} {title}", font=("Segoe UI", 13,
"bold"),
```

```
bg=CARD_BG, fg=TEXT_PRI).pack(side="left")
```

```
pri_color = PRIORITY_CLR.get(priority, "#64748b")
```

```
badge = tk.Label(title_row, text=f" {priority} ",
```

```
font=("Segoe UI", 9, "bold"),
```

```
bg=pri_color, fg="white", padx=6, pady=2)
```

```
badge.pack(side="right", padx=5)
```

```
tk.Label(body, text=desc, font=("Segoe UI", 11), bg=CARD_BG,
```

```
fg=TEXT_SEC, wraplength=900,
justify="left").pack(anchor="w", pady=(6, 0))
```

```
#
```

```
# DATA STRUCTURES INSPECTOR
```

```
#
```

```
def show_ds_inspector(self):
```

```
self.clear_main()
```

```
self._page_header("Data Structures Inspector", "Live view into all active
data structures powering this app")
```

```
tabview = ctk.CTkTabview(self.main, fg_color=CARD_BG,
```

```
segmented_button_fg_color="#0d1627",
```

```
segmented_button_selected_color=ACCENT,
```

```

segmented_button_selected_hover_color="#00b390"
,
segmented_button_unselected_color="#0d1627",
segmented_button_unselected_hover_color="#15204
0",

text_color=TEXT_PRI)

tabview.pack(fill="both", expand=True, padx=30, pady=(0, 25))

tabview.add("Doubly Linked List")
tabview.add("Stack (LIFO Undo)")
tabview.add("Hash Table O(1)")
tabview.add("Binary Search Tree")
tabview.add("Priority Queue")

def _textbox(parent, content):
    tb = ctk.CTkTextbox(parent, font=ctk.CTkFont(family="Consolas",
size=11),

fg_color=CARD_BG2, text_color=TEXT_PRI,
scrollbar_button_color=BORDER)

    tb.pack(fill="both", expand=True, padx=8, pady=8)

    tb.insert("1.0", content)

    tb.configure(state="disabled")

# ——— Doubly Linked List

dll_content = (f"Head →
{self.data_manager.dll.head.data['collection_id'] if self.data_manager.dll.head
else 'NULL'}\n"

```

```

        f"Tail →
{self.data_manager.dll.tail.data['collection_id'] if self.data_manager.dll.tail
else 'NULL'}\n"

        f"Size = {self.data_manager.dll.size}\n\n")

curr = self.data_manager.dll.head

idx = 0

while curr:

    prv = curr.prev.data['collection_id'] if curr.prev else "NULL"
    nxt = curr.next.data['collection_id'] if curr.next else "NULL"

    dll_content +=
(f"[{idx}] ({prv} ← {curr.data['collection_id']} → {nxt})\n"

        f"        Resident : {curr.data['resident']}\n"
        f"        Area      : {curr.data['area']}\n"
        f"        Total      : {curr.data['total_kg']} kg\n\n")

    curr = curr.next

    idx += 1

_textbox(tabview.tab("Doubly Linked List"), dll_content)

# ——— Stack

items = self.data_manager.undo_stack.to_list()

stack_content = f"Stack Size: {self.data_manager.undo_stack.size}\n"

stack_content += f"Top (Most Recent):
{self.data_manager.undo_stack.peek()}\n\n"

if items:

    for i, item in enumerate(items):

        stack_content += f"[{i}] Action:
{item['action']} → {item['record']['collection_id']}
({item['record']['resident']})\n"

```

```

else:
    stack_content += "Stack is currently empty – no undo history."
    _textbox(tabview.tab("Stack (LIFO Undo)"), stack_content)

# — Hash Table

buckets = self.data_manager.resident_hash.get_buckets_info()

ht_content = f"Total Keys Indexed:
{self.data_manager.resident_hash.size}\n"

ht_content += f"Capacity          :
{self.data_manager.resident_hash.capacity} buckets\n\n"

for bucket_idx, keys in buckets:
    ht_content += f"Bucket [{bucket_idx:02d}]: {' → '.join(keys) if
keys else '(empty)'}\n"

_textbox(tabview.tab("Hash Table O(1)"), ht_content)

# — BST

bst_records = self.data_manager.bst_index.in_order_traversal()

bst_content = f"BST Node Count: {self.data_manager.bst_index.count}\n"

bst_content += "In-Order Traversal (Sorted by Collection ID):\n\n"

for key, record in bst_records:
    bst_content += (f"  Key: '{key}' → Resident:
{record['resident']} "

                    f"| Weight: {record['total_kg']} kg | Area:
{record['area']}\n")

_textbox(tabview.tab("Binary Search Tree"), bst_content)

```

```

# — Priority Queue

heap_items = self.data_manager.pickup_pq.to_list()
top = self.data_manager.pickup_pq.peak()
pq_content = f"Heap Size    : {len(heap_items)}\n"
pq_content += f"Top Dispatch: {top['collection_id']} if top else
'None'}\n"
pq_content += f"Strategy    : Higher total weight = lower priority score
= dispatched first\n\n"

for priority, data in heap_items:
    pq_content += (f"    Score
[{priority:03d}]  →  {data['collection_id']} |  "
                  f"Area: {data['area']} |  Weight: {data['total_kg']}
kg\n")

    _textbox(tabview.tab("Priority Queue"), pq_content)

#

# UNDO

#

def undo_action(self):
    success, msg = self.data_manager.undo_last_action()

    if success:
        messagebox.showinfo("Undo ↵", msg)

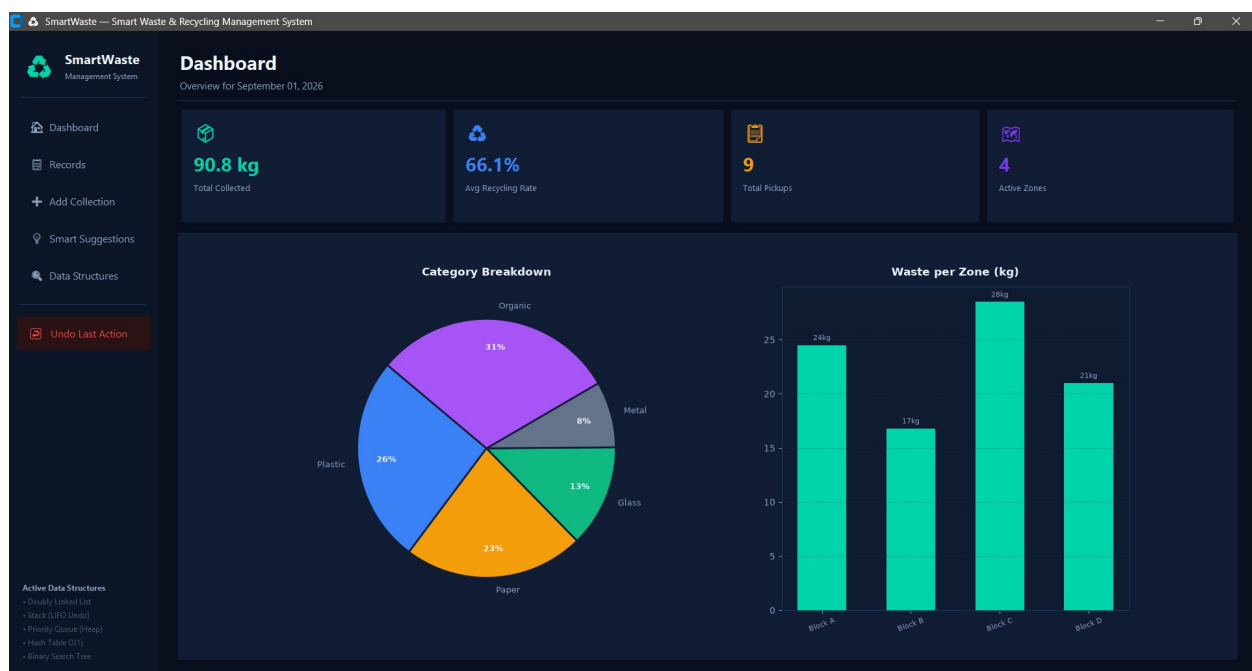
        self.show_dashboard()

    else:
        messagebox.showwarning("Undo ↵", msg)

```

```
# ——— Entry Point
```

```
if __name__ == "__main__":  
    app = SmartWasteApp()  
    app.mainloop()
```



SmartWaste
Management System

Dashboard

Records

+ Add Collection

Smart Suggestions

Data Structures

Undo Last Action

Active Data Structures

• Doubly Linked List

• Stack (LIFO Undo)

• Priority Queue (Heap)

• Hash Table (O(1))

• Binary Search Tree

Records Manager

Browse, search and review all waste collection records

ID	Resident	Zone	Date	Plastic	Paper	Glass	Metal	Organic	Total	Rate
WC001	Arun Kumar	Block A	2026-08-25	2.5	1.5	1.0	0.5	3.0	8.5	64.7%
WC002	Priya	Block B	2026-08-26	1.2	2.8	0.5	0.8	4.5	9.8	54.1%
WC003	Karthik	Block C	2026-08-27	4.0	3.0	2.5	1.5	2.0	13.0	84.6%
WC004	Meena	Block A	2026-08-28	0.8	1.0	0.0	0.2	5.0	7.0	28.6%
WC005	Rahul	Block D	2026-08-29	3.5	2.0	1.8	1.2	3.5	12.0	70.8%
WC006	Divya	Block B	2026-08-30	2.0	1.5	0.8	0.5	2.2	7.0	68.6%
WC007	Sanjay	Block C	2026-08-31	5.0	4.0	3.0	2.0	1.5	15.5	90.3%
WC008	Anitha	Block D	2026-09-01	1.5	2.2	1.0	0.3	4.0	9.0	55.6%
WC009	Vera Chen	Block A	2026-09-02	3.0	2.5	1.0	0.5	2.0	9.0	77.8%

Search

Clear

SmartWaste
Management System

Dashboard

Records

+ Add Collection

Smart Suggestions

Data Structures

Undo Last Action

Active Data Structures

• Doubly Linked List

• Stack (LIFO Undo)

• Priority Queue (Heap)

• Hash Table (O(1))

• Binary Search Tree

Add New Collection

Log a waste pickup — data is stored across all DS structures

Collection Identity

Collection ID

WC010

Resident / Collector Name

Municipal Zone

Zone A (Block 1)

Collection Date (YYYY-MM-DD)

2026-09-01

Notes / Remarks (optional)

Save Collection Record

Clear Form

Waste Breakdown (kg)

Enter the weight collected per category below:

Plastic

0.0

kg

Paper

0.0

kg

Glass

0.0

kg

Metal

0.0

kg

Organic

0.0

kg

Total: 0.0 kg

SmartWaste
Management System

Dashboard

Records

Add Collection

Smart Suggestions

Data Structures

Undo Last Action

Active Data Structures

Doubly Linked List

Stack (LIFO Undo)

Priority Queue (Heap)

Hash Table O(1)

Binary Search Tree

Smart Suggestions

AI-driven eco tips and municipal recommendations based on your data

Paper Recycling Program

22.6% paper waste detected. Partner with recycling mills — every tonne of recycled paper saves 17 trees and 26,000 litres of water.

Medium

Schedule Regular Audits

Monthly waste data audits help identify area-wise trends early. Use the Dashboard Charts to compare zone performance and take proactive action.

General

Digital Reporting & Resident Awareness

Send monthly recycling reports to residents. Communities with regular feedback reduce waste generation by an average of 22% within 6 months.

General

SmartWaste
Management System

Dashboard

Records

Add Collection

Smart Suggestions

Data Structures

Undo Last Action

Active Data Structures

Doubly Linked List

Stack (LIFO Undo)

Priority Queue (Heap)

Hash Table O(1)

Binary Search Tree

Data Structures Inspector

Live view into all active data structures powering this app

Doubly Linked List

Stack (LIFO Undo)

Hash Table O(1)

Binary Search Tree

Priority Queue

Head → WC001

Tail → WC009

Size → 9

[0] (NULL ← WC001 → WC002)

Resident : Arun Kumar

Area : Block A

Total : 8.5 kg

[1] (WC001 ← WC002 → WC003)

Resident : Priya

Area : Block B

Total : 9.8 kg

[2] (WC002 ← WC003 → WC004)

Resident : Earthik

Area : Block C

Total : 13.8 kg

[3] (WC003 ← WC004 → WC005)

Resident : Meena

Area : Block A

Total : 7.8 kg

[4] (WC004 ← WC005 → WC006)

Resident : Rahul

Area : Block D

Total : 12.8 kg

[5] (WC005 ← WC006 → WC007)

Resident : Divya

Area : Block B

Total : 7.8 kg

[6] (WC006 ← WC007 → WC008)

Resident : Sanjay

Area : Block C

Total : 15.5 kg

[7] (WC007 ← WC008 → WC009)

Resident : Anitha

Area : Block D

Total : 9.8 kg

SmartWaste

Management System

Dashboard

Records

+ Add Collection

Smart Suggestions

Data Structures

Undo Last Action

Active Data Structures

Doubly Linked List

Stack (LIFO Undo)

Hash Table O(1)

Binary Search Tree

Priority Queue

Data Structures Inspector

Live view into all active data structures powering this app

Doubly Linked List

Stack (LIFO Undo)

Hash Table O(1)

Binary Search Tree

Priority Queue

Area : Block A

Total : 8.5 kg

[1] (WC001 + WC002 + WC003)

Resident : Priya

Area : Block B

Total : 9.8 kg

[2] (WC002 + WC003 + WC004)

Resident : Karthik

Area : Block C

Total : 13.0 kg

[3] (WC003 + WC004 + WC005)

Resident : Meena

Area : Block A

Total : 7.0 kg

[4] (WC004 + WC005 + WC006)

Resident : Rahul

Area : Block D

Total : 12.0 kg

[5] (WC005 + WC006 + WC007)

Resident : Divya

Area : Block B

Total : 7.0 kg

[6] (WC006 + WC007 + WC008)

Resident : Sanjay

Area : Block C

Total : 15.5 kg

[7] (WC007 + WC008 + WC009)

Resident : Anitha

Area : Block D

Total : 9.0 kg

[8] (WC008 + WC009 + NULL)

Resident : Vera Chen

Area : Block A

Total : 9.0 kg